

SAST

Use case: Sign in

Abuse case: Unauthorized Access

Security case: Implement Multi-Factor Authentication

```

@ Appointments.java  @ userLogin.java x
18 public class userLogin { 16 usages 1 RuqayaAbdulhussein
55 private void authenticate() { 1 usage 1 RuqayaAbdulhussein
92 String query = "SELECT password, salt FROM Users WHERE id=?";
93
94 try {
95 PreparedStatement statement = con.prepareStatement(query);
96 statement.setString( parameterIndex: 1, userID);
97 ResultSet rs = statement.executeQuery();
98
99 if (rs.next()) {
100 String storedHashedPassword = rs.getString( columnLabel: "password");
101 String storedSalt = rs.getString( columnLabel: "salt");
102 byte[] hashedSalt = hexStringToByteArray(storedSalt);
103
104 String hashedPassword = generateHash(password, hashedSalt);
105
106 if (hashedPassword.equals(storedHashedPassword)) { //hash inputted pass and compare it with the one in the database
107 failedAttempts = 0;
108 enterOTP enterotp = new enterOTP(stage, userID);
109 enterotp.initializeComponents();
110 }
111 } else {
112 lastFailedAttempt = System.currentTimeMillis();
113 showAlert( title: "Authentication Failed", content: "Invalid ID or password.");
114 }
115 } else {
116 failedAttempts++;
117 lastFailedAttempt = System.currentTimeMillis();
118 showAlert( title: "Authentication Failed", content: "User not found.");
119 }
120
121 DBUtils.closeConnection(con, statement);
122 } catch (Exception e) {
123 e.printStackTrace();
124 showAlert( title: "Database Error", content: "Failed to connect to the database.");
125 }
126 }
127 }

@ enterOTP.java x
12 public class enterOTP { 2 usages 1 RuqayaAbdulhussein
89 private void verifyOTP() { 1 usage 1 RuqayaAbdulhussein
104 resultSet rs = statement.executeQuery();
105
106 if (rs.next()) {
107 String email = rs.getString( columnLabel: "email");
108 String role = rs.getString( columnLabel: "role");
109 Timestamp lastOtpTime = rs.getTimestamp( columnLabel: "last_otp_time");
110
111 if (lockTime != 0) {
112 long lockDuration = System.currentTimeMillis() - lockTime;
113 if (lockDuration < 60000) { //if its been less than 60 seconds since the user got locked they will be prompted to wait
114 showAlert( title: "Locked Out", content: "Too many failed attempts. Try again in 1 minute.");
115 return;
116 } else {
117 resetFailedAttempts(); //failed attempts reset after 60 seconds and they can try again
118 }
119 }
120
121 if (lastOtpTime == null || (System.currentTimeMillis() - lastOtpTime.getTime()) > 60000) { //if its been longer than 60 seconds since the last otp, the user must request a new
122 showAlert( title: "Expired OTP", content: "Your OTP has expired. Please request a new one.");
123 return;
124 }
125
126 if (OTPStorage.verifyOTP(email, enteredOTP)) { //verify email and otp in the otp storage
127 resetFailedAttempts();
128 statusLabel.setText("OTP Verified. Login Successful!");
129 redirectUser(role, userID);
130 } else {
131 failedAttempts++;
132 if (failedAttempts >= 3) {
133 lockTime = System.currentTimeMillis();
134 showAlert( title: "Locked Out", content: "Too many failed attempts. Try again in 1 minute.");
135 } else {
136 showAlert( title: "Incorrect OTP", content: "Wrong OTP. Attempts left: " + (3 - failedAttempts));
137 }
138 }
139 } else {
140 showAlert( title: "Authentication Failed", content: "Invalid ID or password.");
141 }
142 }
143 }

```

Upon logging in, credentials are validated against user input and the database otherwise they can be locked out for failed attempts, only then they are directed for MFA which is also validated against input validation and attempt limiting.

Use case: Change Password

```
changePassword.java x
18 public class changePassword { 6 usages  RuqayaAbdulhussein
74 private void changePassword() throws NoSuchAlgorithmException { 1 usage  RuqayaAbdulhussein
75     String oldPassword = oldPasswordField.getText();
76     String newPassword = newPasswordField.getText();
77
78     if (oldPassword.isEmpty() || newPassword.isEmpty()) { //ensures both fields are filled
79         showAlertError( title: "Input Error", content: "Both fields must be filled.");
80         return;
81     }
82
83     Connection con = DBUtils.establishConnection();
84     String query = "SELECT password, salt FROM Users WHERE id=?";
85
86     try (PreparedStatement preparedStatement = con.prepareStatement(query)) {
87         preparedStatement.setString( parameterIndex: 1, userID);
88         ResultSet rs = preparedStatement.executeQuery();
89
90         if (rs.next()) {
91             //regular expression to ensure new password entered meets compexlity requirments
92             String passRegex = "^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[@#%!*_-+=~]).{8,}$";
93
94             Pattern passPattern = Pattern.compile(passRegex);
95
96             Matcher matchnewPass = passPattern.matcher(newPassword);
97             if (matchnewPass.matches()){
98                 String storedHashedPassword = rs.getString( columnLabel: "password");
99                 String storedSalt = rs.getString( columnLabel: "salt");
100
101                 String hashedOldPassword = generateHash(oldPassword, algorithm: "SHA-256", hexStringToByteArray(storedSalt));
102
103                 if (storedHashedPassword.equals(hashedOldPassword)) { //ensures old password is correct
104                     byte[] newSalt = createSalt();
105                     String newSaltString = bytesToStringHex(newSalt);
106                     String hashedNewPassword = generateHash(newPassword, algorithm: "SHA-256", newSalt);
107
108                     String updateQuery = "UPDATE Users SET password=?, salt=? WHERE id=?";
109                     try (PreparedStatement updateStatement = con.prepareStatement(updateQuery)) {
```

The code shows that the new password is verified to meet password complexity requirements, as well as verification of the old password before implementing the new change.

Use case: Manage User Accounts

Abuse case: Unauthorized modification or deletion of user accounts

Security Case: Implement RBAC and logging

```
12 public class enterOTP { 2 usages 1 RuqayaAbdullahussein *
13 //this function will redirect the user to the correct page based on their role (RBAC)
14 private void redirectUser(String role, String userID) { 1 usage 1 RuqayaAbdullahussein *
15 String query = "SELECT Firstname, lastname FROM Users WHERE id = ?";
16 String userFN = null;
17 String userLN = null;
18
19 try {Connection con = DBUtils.establishConnection();
20 PreparedStatement statement = con.prepareStatement(query)} {
21 statement.setString(parameterIndex: 1, userID);
22 ResultSet rs = statement.executeQuery();
23
24 if (rs.next()) {
25 userFN = rs.getString(columnLabel: "firstname");
26 userLN = rs.getString(columnLabel: "lastname");
27 }
28 DBUtils.closeConnection(con, statement);
29 } catch (SQLException e) {
30 e.printStackTrace();
31 showAlert(title: "Database Error", content: "Failed to fetch user details.");
32 return;
33 }
34
35 if (role.equals("Admin")){
36 userManagement userManage = new userManagement(stage, userID, userFN, userLN);
37 userManage.initializeComponents();
38 }else if (role.equals("Receptionist")){
39 patientRegistration patientreg = new patientRegistration(stage, userID, userFN, userLN);
40 patientreg.initializeComponents();
41 }else if (role.equals("Doctor") || role.equals("Nurse")){
42 patientRecordsDocNurse patientRecordsdoc = new patientRecordsDocNurse(stage, userID, userFN, userLN);
43 patientRecordsdoc.initializeComponents();
44 }else{
45 statusLabel.setText("You do not have permissions to access this system.");
46 }
47 }
48 }
```

RBAC is applied as each user is redirected to the page they can see only based on their role, therefore unauthorized users will not be able to access the user management page unless they are registered in the database as an admin.

```
1 public class userManagement { 4 usages 1 RuqayaAbdullahussein *
2 private void addNewUser() throws SQLException { 1 usage 1 RuqayaAbdullahussein *
3 if (checkDuplicateID(QID)) //verifies the user isn't adding the same user again by verifying the QID
4 return; //if QID is duplicate stop the process
5
6 String newPassWord = firstName.toLowerCase() + " " + lastName.toLowerCase(); //password is created by default for the new user with their first and last name.
7
8 byte[] salt = createSalt();
9 String saltString = bytesToStringHex(salt);
10
11 String hashedPassword = generateHash(newPassWord, salt); //hash the default password before storing it into the database along with the salt
12 //inserting into the database using parameterized queries
13 String adminName = userFN + " " + userLN;
14 String sql = "INSERT INTO Users (QID, Firstname, lastname, email, phone, role, specialization, department, salt, password, created_by, last_updated_by) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
15
16 try {Connection con = DBUtils.establishConnection();
17 PreparedStatement statement = con.prepareStatement(sql)} {
18 statement.setString(parameterIndex: 1, QID);
19 statement.setString(parameterIndex: 2, firstName);
20 statement.setString(parameterIndex: 3, lastName);
21 statement.setString(parameterIndex: 4, email);
22 statement.setString(parameterIndex: 5, phone);
23 statement.setString(parameterIndex: 6, adminName.getSelectedRole());
24 statement.setString(parameterIndex: 7, adminName.getSelectedSpec());
25 statement.setString(parameterIndex: 8, adminName.getSelectedDep());
26 statement.setString(parameterIndex: 9, saltString);
27 statement.setString(parameterIndex: 10, hashedPassword);
28 statement.setString(parameterIndex: 11, adminName);
29 statement.setString(parameterIndex: 12, null);
30 }
```

```
1 public class userManagement { 4 usages 1 RuqayaAbdullahussein *
2 private void updateUser(ActionEvent event) { 1 usage 1 RuqayaAbdullahussein *
3 if (selectedUser != null) {
4 Alert confirmDeleteAlert = new Alert(AlertType.CONFIRMATION); //verifies if the user would like to go through with this update.
5 confirmDeleteAlert.setTitle("Confirm Update");
6 confirmDeleteAlert.setHeaderText("Are you sure you want to update this user?");
7
8 confirmDeleteAlert.showAndWait(); //present ButtonType response -> {
9 if (response == ButtonType.NO) {
10 String sql = "UPDATE Users SET Firstname = ?, Lastname = ?, email = ?, phone = ?, role = ?, specialization = ?, department = ?, last_updated_by = ? WHERE id = ?";
11
12 try {Connection con = DBUtils.establishConnection();
13 PreparedStatement statement = con.prepareStatement(sql)} {
14 statement.setString(parameterIndex: 1, adminpane.firstName.getText());
15 statement.setString(parameterIndex: 2, adminpane.lastName.getText());
16 statement.setString(parameterIndex: 3, adminpane.email.getText());
17 statement.setString(parameterIndex: 4, adminpane.phone.getText());
18 statement.setString(parameterIndex: 5, adminpane.getSelectedRole());
19 statement.setString(parameterIndex: 6, adminpane.getSelectedSpec());
20 statement.setString(parameterIndex: 7, adminpane.getSelectedDep());
21 statement.setString(parameterIndex: 8, adminName);
22 statement.setString(parameterIndex: 9, selectedUser.getId());
23
24 statement.executeUpdate();
25 clearFields();
26 loadUserData();
27
28 showAlert(title: "Success", content: "User updates successfully.");
29 DBUtils.closeConnection(con, statement);
30 } catch (SQLException e) {
31 e.printStackTrace();
32 showAlert(title: "Error", content: "Failed to update the user.");
33 }
34 }
35 } else {
36 showAlert(title: "Error", content: "No user selected to be updated.");
37 }
```

In, addition, there is logging implemented for any addition or updating of users through the logged in users' first and last name.

Use case: Register New Patient

```
171 public class patientRegistration { 4 usages 4 RuyyashAbdullahussain
172     private void addPatient() throws NoSuchElementException { 1 usage 4 RuyyashAbdullahussain
173         if (!Validation()) {
174             return;
175         }
176         String receptionistName = userFW + " " + userLS;
177         String QID = receptionistPanel.QID.getText().trim();
178         String firstName = receptionistPanel.firstName.getText().trim();
179         String lastName = receptionistPanel.lastName.getText().trim();
180         String phone = receptionistPanel.phone.getText().trim();
181         String email = receptionistPanel.email.getText().trim();
182         String emergencyContact = receptionistPanel.emergencyContact.getText().trim();
183
184         if (!checkDuplicateID(QID)) {
185             return;
186         }
187
188         String sql = "INSERT INTO Patients (receptionist_id, QID, firstName, lastName, date_of_birth, gender, blood_type, nationality, phone, email, emergency_contact, " +
189             "emergency_contact_relationship, created_by, last_updated_by) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
190
191         try (Connection con = DBUtils.establishConnection();
192             PreparedStatement statement = con.prepareStatement(sql)) {
193             statement.setString(1, receptionistName);
194             statement.setString(2, Long.parseLong(QID));
195             statement.setString(3, firstName);
196             statement.setString(4, lastName);
197             LocalDate Dob = receptionistPanel.dateOfBirth.getValue();
198             statement.setDate(5, java.sql.Date.valueOf(Dob));
199             statement.setString(6, receptionistPanel.getSelectedGender());
200             statement.setString(7, receptionistPanel.getSelectedBloodType());
201             statement.setString(8, receptionistPanel.getSelectedNationality());
202             statement.setString(9, phone);
203             statement.setString(10, email);
204             statement.setString(11, emergencyContact);
205             statement.setString(12, receptionistPanel.getSelectedRS());
206             statement.setString(13, receptionistName);
207             statement.setString(14, null);
208         }
```

Although there are no abuse cases specified, registering a new user still has RBAC upon logging in (must be a receptionist), input validation, and parameterized queries implemented.

Use case: Assign a doctor to a patient

Abuse case: Appointment Overbooking

Security case: Rate Limiting

```
171 public class Appointments { 2 usages 4 RuyyashAbdullahussain
172     private void addAppointment() throws NoSuchElementException { 1 usage 4 RuyyashAbdullahussain
173         if (!Validation()) {
174             return;
175         }
176         String purposeOfVisit = appointmentPanel.POV.getText().trim();
177         String receptionistName = userFW + " " + userLS;
178         String doctorName = appointmentPanel.getSelectedDoctor();
179         LocalDate appointmentDate = appointmentPanel.dateOfAppointment.getValue();
180         String appointmentTime = appointmentPanel.getSelectedTime();
181
182         if (!checkDuplicateID(patientID, doctorName, appointmentDate, appointmentTime)) { // prevents receptionist from creating several appointments that are the same
183             //this is also a mitigation for appointment overbooking (DoS)
184             showAlertError("Duplicate Appointment", content: "An appointment with this doctor at the same time already exists.");
185             return;
186         }
187
188         try (Connection con = DBUtils.establishConnection();
189             PreparedStatement statement = con.prepareStatement(sql)) {
190             statement.setString(1, userID);
191             statement.setString(2, patientID);
192             statement.setString(3, appointmentPanel.getSelectedDoctor());
193             LocalDate DateOfApp = appointmentPanel.dateOfAppointment.getValue();
194             statement.setDate(4, java.sql.Date.valueOf(DateOfApp));
195             statement.setString(5, appointmentPanel.getSelectedTime());
196             statement.setString(6, appointmentPanel.getSelectedDepartment());
197             statement.setString(7, purposeOfVisit);
198             statement.setString(8, appointmentPanel.getSelectedStatus());
199             statement.setString(9, receptionistName);
200             statement.setString(10, null);
201
202             statement.executeUpdate();
203             clearFields();
204             loadAppointmentData(patientID);
205             DBUtils.closeConnection(con, statement);
206         }
207     }
208
209     private void clearFields() {
210         appointmentPanel.doctorName.setValue(null);
211         appointmentPanel.dateOfAppointment.setValue(null);
212         appointmentPanel.time.setValue(null);
213         appointmentPanel.department.setValue(null);
214         appointmentPanel.POV.clear();
215         appointmentPanel.status.setValue(null);
216     }
217
218     private boolean checkDuplicateID(String patientID, String doctorName, LocalDate date, String time) { 1 usage 4 RuyyashAbdullahussain
219         String sql = "SELECT COUNT(*) FROM Appointments WHERE patient_id = ? AND doctor_name = ? AND date = ? AND time = ?";
220
221         try (Connection con = DBUtils.establishConnection();
222             PreparedStatement statement = con.prepareStatement(sql)) {
223             statement.setString(1, patientID);
224             statement.setString(2, doctorName);
225             statement.setDate(3, java.sql.Date.valueOf(date));
226             statement.setString(4, time);
227
228             ResultSet rs = statement.executeQuery();
229             if (rs.next() && rs.getInt(1) > 0) {
230                 return true;
231             }
232             DBUtils.closeConnection(con, statement);
233         } catch (SQLException e) {
234             e.printStackTrace();
235             showAlertError("Database Error", content: "Failed to check for duplicate appointments.");
236         }
237         return false;
238     }
239 }
```

The code shows rate limiting implemented upon booking an appointment by checking for any duplicates. This would prevent an attacker from overloading the system with more appointments than a doctor can handle in a day. As well as logging for whoever adds/updates appointments based on the current logged in users name.

Use case: View Update Medical Records

Abuse case: View Patient/Medical Records (SQLi)

Security case: Implement RBAC and use parameterized queries.

```
@enterOTP.java x
12 public class enterOTP { 2 usages 1 RugayaAbdhussein
149
150 > private void resetFailedAttempts() { failedAttempts = 0; }
153
154 //this function will redirect the user to the correct page based on their role (RBAC)
155 private void redirectUser(String role, String userID){ 1 usage 1 RugayaAbdhussein
156     String query = "SELECT firstname, lastname FROM Users WHERE id = ?";
157     String userFN = null;
158     String userLN = null;
159
160     try (Connection con = DBUtils.establishConnection();
161         PreparedStatement statement = con.prepareStatement(query)) {
162         statement.setString( parameterIndex: 1, userID);
163         ResultSet rs = statement.executeQuery();
164
165         if (rs.next()) {
166             userFN = rs.getString( columnLabel: "firstname");
167             userLN = rs.getString( columnLabel: "lastname");
168         }
169         DBUtils.closeConnection(con, statement);
170     } catch (SQLException e) {
171         e.printStackTrace();
172         showAlert( title: "Database Error", content: "Failed to fetch user details.");
173         return;
174     }
175
176     if (role.equals("Admin")){
177         userManagement userManager = new userManagement(stage, userID, userFN, userLN);
178         userManager.initializeComponents();
179     }else if (role.equals("Receptionist")){
180         patientRegistration patientreg = new patientRegistration(stage, userID, userFN, userLN);
181         patientreg.initializeComponents();
182     }else if (role.equals("Doctor") || role.equals("Nurse")){
183         patientRecordsDocNurse patientRecordsDocNurse = new patientRecordsDocNurse(stage, userID, userFN, userLN);
184         patientRecordsDocNurse.initializeComponents();
185     }else {
186         // Extract Surround //
187         // have permissions to access this system."});
188     }
189 }
```

```
@enterOTP.java @userLogin.java
18 public class userLogin { 16 usages 1 RugayaAbdhussein
55 private void authenticate() { 1 usage 1 RugayaAbdhussein
58
59     String userIDRegex = "^[0-9]{4}$"; //validation to ensure user ID is only 4 digits
60     Pattern idPattern = Pattern.compile(userIDRegex);
61     Matcher matchID = idPattern.matcher(userID);
62
63     String passRegex = "^[a-zA-Z0-9@#%*~\\-+=-]{8}$";
64     Pattern passPattern = Pattern.compile(passRegex);
65     Matcher matchPass = passPattern.matcher(password);
66
67     if (userID.isEmpty() || !matchID.matches()) {
68         failedAttempts++;
69         lastFailedAttempt = System.currentTimeMillis();
70         showAlert( title: "Validation Error", content: "User ID must be a 4-digit number.");
71         return;
72     }
73
74     if (password.isEmpty() || !matchPass.matches()) {
75         failedAttempts++;
76         lastFailedAttempt = System.currentTimeMillis();
77         showAlert( title: "Validation Error", content: "Password field must meet complexity requirements");
78         return;
79     }
80
81     Connection con = DBUtils.establishConnection();
82     String query = "SELECT password, salt FROM Users WHERE id=?";
83
84     try {
85         PreparedStatement statement = con.prepareStatement(query);
86         statement.setString( parameterIndex: 1, userID);
87         ResultSet rs = statement.executeQuery();
88
89         if (rs.next()) {
90             String storedHashedPassword = rs.getString( columnLabel: "password");
91             String storedSalt = rs.getString( columnLabel: "salt");
92             byte[] hashedSalt = hexStringToByteArray(storedSalt);
93
94             String hashedPassword = generateHash(password, hashedSalt);
95         }
96     }
97 }
```

```
private void verifyOTP() { 1 usage 1 RugayaAbdhussein
    Connection con = DBUtils.establishConnection();
    String query = "SELECT email, role, last_otp_time FROM Users WHERE id=?";

    String enteredOTP = otpField.getText();

    String otpRegex = "^[0-9]{6}$"; //input validation to ensure otp entered is 6 digits.
    if (!enteredOTP.matches(otpRegex)) {
        showAlert( title: "Invalid OTP", content: "OTP must be a 6-digit number.");
        return;
    }

    try {
        PreparedStatement statement = con.prepareStatement(query);
        statement.setString( parameterIndex: 1, userID);
        ResultSet rs = statement.executeQuery();

        if (rs.next()) {
            String email = rs.getString( columnLabel: "email");
            String role = rs.getString( columnLabel: "role");
            Timestamp lastOtpTime = rs.getTimestamp( columnLabel: "last_otp_time");
        }
    }
}
```

```

public class patientRecordsDocNurse { 6 usages  RuqayaAbdulhussein *
    private void medicalRecords() 1 usage  RuqayaAbdulhussein *
    {
        try (Connection con = DBUtils.establishConnection();
            PreparedStatement statement = con.prepareStatement(query)) {
            statement.setString( parameterIndex: 1, userID);
            ResultSet rs = statement.executeQuery();

            if (rs.next()) {
                userRole = rs.getString( columnLabel: "role");
                userFN = rs.getString( columnLabel: "firstname");
                userLN = rs.getString( columnLabel: "lastname");
            }

            DBUtils.closeConnection(con, statement);
        } catch (SQLException e) {
            e.printStackTrace();
            showAlertError( title: "Database Error", content: "Failed to fetch user details.");
            return;
        }

        if (userRole == null) {
            showAlertError( title: "Access Denied", content: "User role not found.");
            return;
        }

        String patientId = selectedPatient.getPatientId();

        if (userRole.equalsIgnoreCase( anotherString: "Doctor")) {
            medicalRecordsDoc medicalrecords = new medicalRecordsDoc(stage, patientId, userID, userFN, userLN);
            medicalrecords.initializeComponents();
        } else if (userRole.equalsIgnoreCase( anotherString: "Nurse")) {
            medicalRecordsNurse medicalrecords = new medicalRecordsNurse(stage, patientId, userID, userFN, userLN);
            medicalrecords.initializeComponents();
        } else {
            showAlertError( title: "Access Denied", content: "You do not have permission to access medical records.");
        }
    }
}

```

SQL injections used to view and update patient records are mitigated through the parameterized queries and input validation in both the login and MFA entry points. In additions to role-based access control which is checked when the user logs in, and then again upon navigating to the medical records.

Use case: Prescribe Medication

Abuse case: Prescription Fraud

Security case: Implement RBAC and Logging

```
private void medicalRecords() { 1 usage  ± RuqayaAbdulhussein *

    String patientId = selectedPatient.getPatientId();

    if (userRole.equalsIgnoreCase( anotherString: "Doctor")) {
        medicalRecordsDoc medicalrecords = new medicalRecordsDoc(stage, patientId, userID, userFN, userLN);
        medicalrecords.initializeComponents();
    } else if (userRole.equalsIgnoreCase( anotherString: "Nurse")) {
        medicalRecordsNurse medicalrecords = new medicalRecordsNurse(stage, patientId, userID, userFN, userLN);
        medicalrecords.initializeComponents();
    } else {
        showAlertError( title: "Access Denied", content: "You do not have permission to access medical records.");
    }
}
```

```
String nurseId = getNurseId(nurseFirstName, nurseLastName); //Fetch the nurse id

//this will insert the inputted details as well as the doctors name that created this record for logging purposes
String sql = "INSERT INTO MedicalRecords (patient_id, nurse_id, nurse_name, doctor_id, doctor_name, diagnosis, treatment, created_by, last_updated_by) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)";

try (Connection con = DBUtils.establishConnection();
    PreparedStatement statement = con.prepareStatement(sql)) {
    statement.setString( parameterIndex: 1, patientID);
    statement.setString( parameterIndex: 2, nurseID);
    statement.setString( parameterIndex: 3, doctorPanel.getSelectedNurse());
    statement.setString( parameterIndex: 4, userID);
    statement.setString( parameterIndex: 5, doctorName);
    statement.setString( parameterIndex: 6, diagnosis);
    statement.setString( parameterIndex: 7, treatment);
    statement.setString( parameterIndex: 8, doctorName);
    statement.setString( parameterIndex: 9, null);

    statement.executeUpdate();
    clearFields();
    loadAppointmentData(patientID);
    DBUtils.closeConnection(con, statement);
}
```

```
confirmUpdateAlert.showAndWait().ifPresent( ButtonType response -> {
    if (response == ButtonType.OK) { //updates user information and logs the doctors name for last updated by
        String sql = "UPDATE MedicalRecords SET patient_id = ?, nurse_id = ?, nurse_name = ?, diagnosis = ?, treatment = ?, last_updated_by = ? WHERE record_id = ?";

        try (Connection con = DBUtils.establishConnection();
            PreparedStatement statement = con.prepareStatement(sql)) {
            statement.setString( parameterIndex: 1, patientID);
            statement.setString( parameterIndex: 2, nurseID);
            statement.setString( parameterIndex: 3, doctorPanel.getSelectedNurse());
            statement.setString( parameterIndex: 4, doctorPanel.diagnosis.getText());
            statement.setString( parameterIndex: 5, doctorPanel.treatment.getText());
            statement.setString( parameterIndex: 6, doctorName);
            statement.setInt( parameterIndex: 7, selectedRecord.getRid());
        }
    }
}
```

```
private void updateAppointment(ActionEvent event) { 1 usage  ± RuqayaAbdulhussein

    String doctorName = userFN + " " + userLN;
    if (!Validation()) {
        return;
    }
    MedicalRecordsData selectedRecord = tableView.getSelectionModel().getSelectedItem();
    if (selectedRecord != null) {
        if (!isAuthorizedToEdit(selectedRecord)) {
            showAlertError( title: "Unauthorized", content: "You are not authorized to edit this record.");
            return;
        }
    }
    selectedNurse();
}
```

```

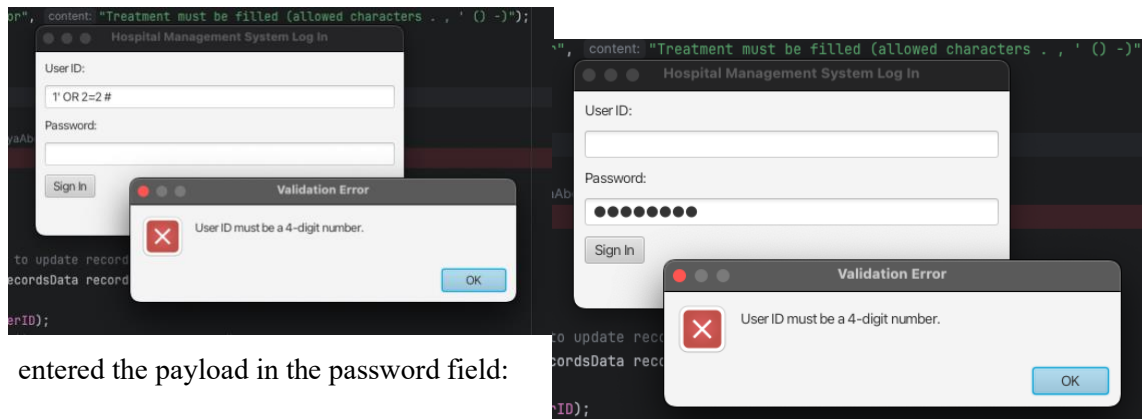
//this method prevents other doctors/nurses to update records which they were not a part of.
private boolean isAuthorizedToEdit(MedicalRecordsData record) { 1 usage ± RuqayaAbdulhussein
    try {
        int userIdInt = Integer.parseInt(userID);
        return userIdInt == record.getDid() || userIdInt == record.getNid();
    } catch (NumberFormatException e) {
        e.printStackTrace();
        return false;
    }
}

```

RBAC is implemented upon prescribing medication as only doctors are redirected the medical records page which allows them to add treatment, whereas the nurse is directed to a page which does not give them the option to edit the “treatment” section. Also, the system checks for who is updating a certain record, only the nurse and doctor who created that record can update it. In addition, based on the logged in user, any additions or updates are logged in the table in which they added or updated.

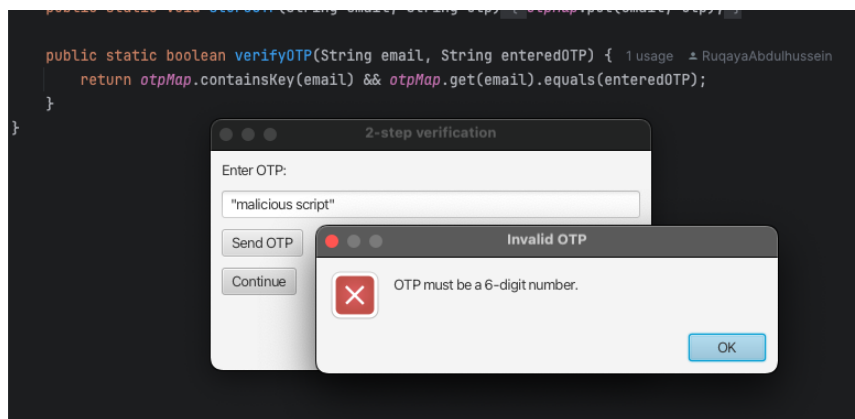
DAST

User Sign In



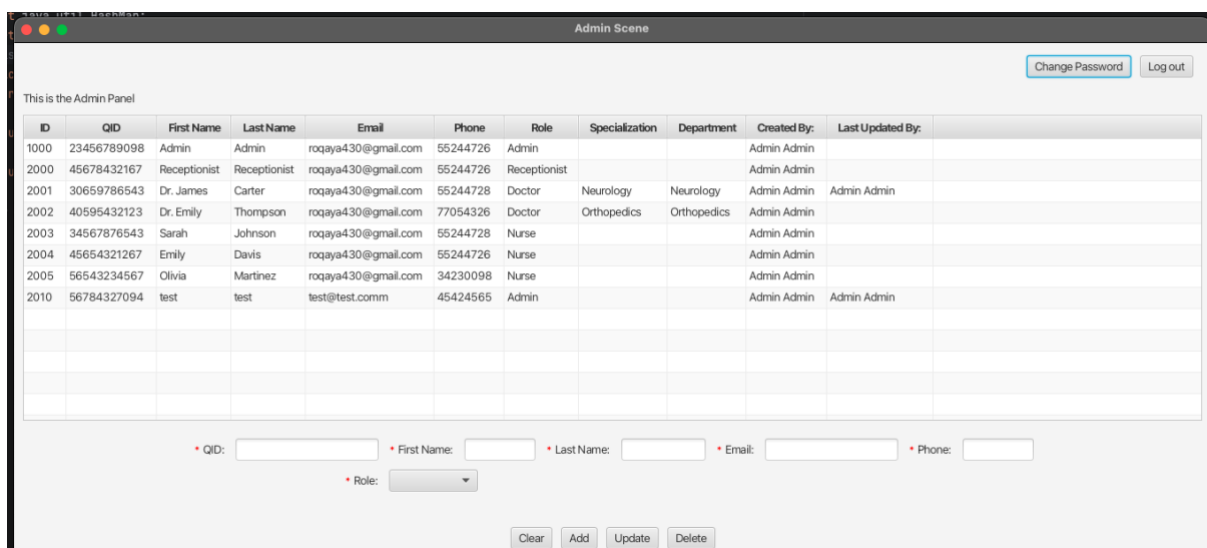
entered the payload in the password field:

‘ OR 2=2



Manage User Accounts

Upon logging in as a admin, this is what displayed, logging in as another user with a different role displays a different page



When clicking on a user to modify, the QID field is disabled to prevent tampering with registered users

Admin Scene

Change PasswordLog Out

This is the Admin Panel

ID	QID	First Name	Last Name	Email	Phone	Role	Specialization	Department	Created By:	Last Updated By:
1000	23456789098	Admin	Admin	roqaya430@gmail.com	55244726	Admin			Admin Admin	
2000	45678432167	Receptionist	Receptionist	roqaya430@gmail.com	55244726	Receptionist			Admin Admin	
2001	30659786543	Dr. James	Carter	roqaya430@gmail.com	55244728	Doctor	Neurology	Neurology	Admin Admin	Admin Admin
2002	40595432123	Dr. Emily	Thompson	roqaya430@gmail.com	77054326	Doctor	Orthopedics	Orthopedics	Admin Admin	
2003	34567876543	Sarah	Johnson	roqaya430@gmail.com	55244728	Nurse			Admin Admin	
2004	45654321267	Emily	Davis	roqaya430@gmail.com	55244726	Nurse			Admin Admin	
2005	56543234567	Olivia	Martinez	roqaya430@gmail.com	34230098	Nurse			Admin Admin	
2010	56784327094	test	test	test@test.comm	45424565	Admin			Admin Admin	Admin Admin

* QID:

34567876543

* First Name:

Sarah

* Last Name:

Johnson

* Email:

roqaya430@gmail.com

* Phone:

55244726

* Role:

Nurse

Clear

Add

Update

Delete

After updating the phone number, the “Last Updated By” field was logged for the logged in admins first and last name.

ID	QID	First Name	Last Name	Email	Phone	Role	Specialization	Department	Created By:	Last Updated By:
1000	23456789098	Admin	Admin	roqaya430@gmail.com	55244726	Admin			Admin Admin	
2000	45678432167	Receptionist	Receptionist	roqaya430@gmail.com	55244726	Receptionist			Admin Admin	
2001	30659786543	Dr. James	Carter	roqaya430@gmail.com	55244728	Doctor	Neurology	Neurology	Admin Admin	Admin Admin
2002	40595432123	Dr. Emily	Thompson	roqaya430@gmail.com	77054326	Doctor	Orthopedics	Orthopedics	Admin Admin	
2003	34567876543	Sarah	Johnson	roqaya430@gmail.com	55244725	Nurse			Admin Admin	Admin Admin

Register New Patient

[illegible]

Upon signing in as a receptionist, the receptionist page displayed, the fields all have input validation as well as duplication prevention, QID tampering prevention, and logging of who created and updated a patients information.

After trying to add the same user again:

This is the Receptionist Panel

Patient ID	QID	First Name	Last Name	Date of Birth	Gender	Blood Type	Nationality	Phone	Email	Emergency Contact	Emergency Contact Relationship	Created By:	Last Updated By:
100	34560020542	Mohamed	Mohamed	2000-05-11	Male	O-	Saudi Arabian	55244726	mohamed123@example.com	12345678	Sibling	Receptionist Receptionist	Receptionist Receptionist
101	45321985743	Fatima	Ahmedd	1999-11-27	Female	A+	Qatari	34230098	fatima@example.com	32658765	Mother	Receptionist Receptionist	Receptionist Receptionist
102	20596847321	Hasan	Abdulrahman	2002-02-01	Male	B+	Palestinian	77054326	hasan@example.com	55643287	Relative	Receptionist Receptionist	Receptionist Receptionist
103	50432985743	Razan	Issa	1992-07-22	Female	A-	Sudanese	66754345	razan@example.com	70987234	Spouse	Receptionist Receptionist	Receptionist Receptionist
106	65432789432	test	test	2025-04-01	Male	A-	Afghan	56785430	test@test.com	65789543	Father	Receptionist Receptionist	Receptionist Receptionist

QID: 50432985743 First Name: Razan Last Name: Issa Date of Birth: 7/22/1992 Gender: Female Blood Type: A-

Nationality: Sudanese Phone: 66754345 Email: razan@example.com Emergency Contact: 70987234 Emergency Contact Relationship: Spouse

Clear Add Update Appointments

Duplicate QID

A patient with this QID already exists.

OK

After updating the first patients emergency contact, it was logged. Currently there is only one receptionist account which I'm using, this is why all the logs come back with the same receptionist name.

This is the Receptionist Panel

Patient ID	QID	First Name	Last Name	Date of Birth	Gender	Blood Type	Nationality	Phone	Email	Emergency Contact	Emergency Contact Relationship	Created By:	Last Updated By:
100	34560020542	Mohamed	Mohamed	2000-05-11	Male	O-	Saudi Arabian	55244726	mohamed123@example.com	65784328	Sibling	Receptionist Receptionist	Receptionist Receptionist
101	45321985743	Fatima	Ahmedd	1999-11-27	Female	A+				558765	Mother	Receptionist Receptionist	Receptionist Receptionist
102	20596847321	Hasan	Abdulrahman	2002-02-01	Male	B+				43287	Relative	Receptionist Receptionist	Receptionist Receptionist
103	50432985743	Razan	Issa	1992-07-22	Female	A-				87234	Spouse	Receptionist Receptionist	Receptionist Receptionist
106	65432789432	test	test	2025-04-01	Male	A-				89543	Father	Receptionist Receptionist	Receptionist Receptionist

QID: script First Name: Last Name: Date of Birth: 4/15/2025 Gender: Male Blood Type: AB+

Nationality: Bangladeshi Phone: Email: Emergency Contact: Emergency Contact Relationship: Sibling

Clear Add Update Appointments

Success

Patent details updated successfully.

Cancel OK

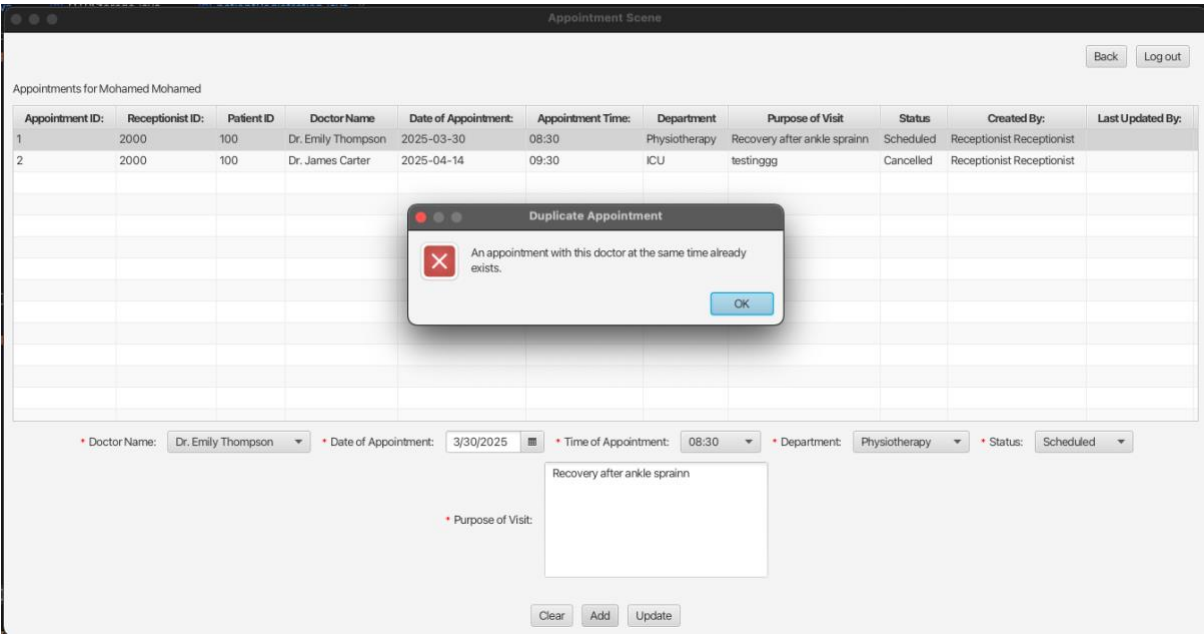
Validation Error

QID must be filled out and a 11-digit number.

OK

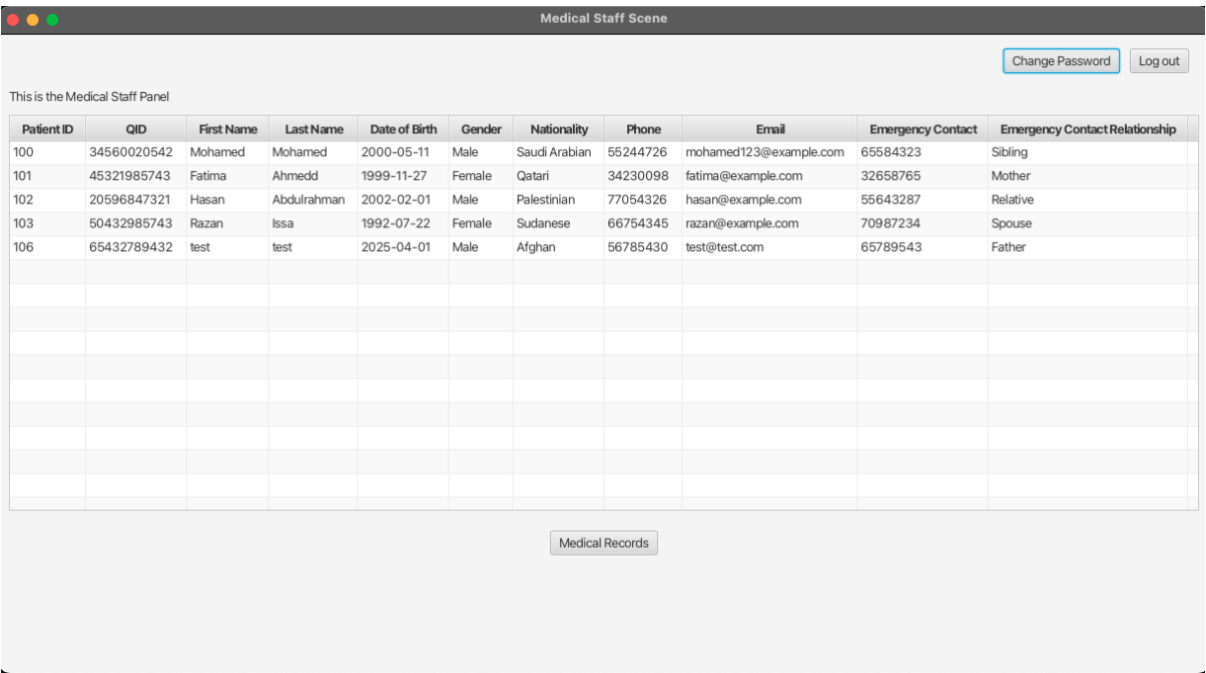
Assign a doctor to patient

Upon viewing the appointments for a patient and attempting to duplicate it, an error message displayed and prevention took place, this prevent over booking the system. In addition, the receptionist was logged after creating an appointment.



View and Update Medical Records

This is what doctors and nurses see when logging in



What doctors see when viewing a patients medical record

Name: Razan Issa

Age: 32

Gender: Female

Blood Type: A-

Nationality: Sudanese

Back

Log out

Record ID:	Patient ID	Nurse ID	Nurse Name	Doctor ID	Doctor Name	Diagnosis:	Treatment:	Created By:	Last Updated By:
1	103	2003	Sarah Johnson	2001	Dr. James Carter	Mild sprain, ankle injury.	Ibuprofen 400 mg Dosage: Take 1 tablet orally every 6-8 hours as needed for pain. Quantity: 30 tablets Instructions: Take with food to prevent stomach irritation. Do not exceed 1200 mg per day.	Dr. James Carter	Dr. James Carter

* Nurse Name:

* Diagnosis:

* Treatment:

Clear

Add

Update

I'm logged in as doctor with ID 2002, so when I tried to update a record, it prevented me since this record is under Doctor with ID 2001 and Nurse with ID 2003, only they can edit.

Name: Razan Issa

Age: 32

Gender: Female

Blood Type: A-

Nationality: Sudanese

Back

Log out

Record ID:	Patient ID	Nurse ID	Nurse Name	Doctor ID	Doctor Name	Diagnosis:	Treatment:	Created By:	Last Updated By:
1	103	2003	Sarah Johnson	2001	Dr. James Carter	Mild sprain, ankle injury.	Ibuprofen 400 mg Dosage: Take 1 tablet orally every 6-8 hours as needed for pain. Quantity: 30 tablets Instructions: Take with food to prevent stomach irritation. Do not exceed 1200 mg per day.	Dr. James Carter	Dr. James Carter

* Nurse Name:

Sarah Johnson

* Diagnosis:

Mild sprain, ankle injury, testing

* Treatment:

Ibuprofen 400 mg
Dosage: Take 1 tablet orally every 6-8 ho
Quantity: 30 tablets

Clear

Add

Update

Unauthorized

You are not authorized to edit this record.

OK

This is what a nurse sees when viewing a patients medical record, the difference is they don't have an option to input the treatment, this is restricted to the doctor only, RBAC is implemented. In additions to logging of who created and last updated a record. Regardless, a nurse cannot update a record since that field is not provided, only a doctor can do so which is logged.

The screenshot shows a web application window titled "Medical Records Scene". At the top, patient details are listed: Name: Razan Issa, Age: 32, Gender: Female, Blood Type: A-, Nationality: Sudanese. Below this is a table with columns: Record ID, Patient ID, Nurse ID, Nurse Name, Doctor ID, Doctor Name, Diagnosis, Treatment, Created By, and Last Updated By. The first row contains data for a record created by Dr. James Carter. Below the table, there are input fields for "Doctor Name" (a dropdown menu) and "Diagnosis" (a text box), along with "Clear", "Add", and "Update" buttons.

Record ID	Patient ID	Nurse ID	Nurse Name	Doctor ID	Doctor Name	Diagnosis	Treatment	Created By	Last Updated By
1	103	2003	Sarah Johnson	2001	Dr. James Carter	Mild sprain, ankle injury.	Ibuprofen 400 mg Dosage: Take 1 tablet orally every 6-8 hours as needed for pain. Quantity: 30 tablets Instructions: Take with food to prevent stomach irritation. Do not exceed 1200 mg per day.	Dr. James Carter	Dr. James Carter

Change Password

The screenshot shows a "Change Password" form with fields for "Old Password" and "New Password", and buttons for "Change Password" and "Back". An "Error" dialog box is displayed over the form, stating: "Password must be at least 8 characters, with a lower and upper case character and at least 1 special character". Below the form, a note reads: "Upon clicking this button, you will be logged out and must log back in with your new password".

The system successfully prevented me from entering the wrong old password, and ensured my new password meets the password complexity requirements.

This screenshot is similar to the previous one, but the "Error" dialog box states: "Old password is incorrect.".